

01_Project introduction:

□ 1. Project Title

Automation Testing of OrangeHRM Demo Web Application Using Selenium WebDriver with TestNG Framework

This project focuses on designing and implementing a scalable automation testing framework for the OrangeHRM demo application using Selenium WebDriver, Java, and TestNG. The framework follows the Page Object Model (POM) design pattern to ensure maintainability, reusability, and readability of test scripts.

□ 2. Project Overview

The OrangeHRM application is a Human Resource Management System used for managing employees, attendance, leave requests, and organizational workflows.

Manual testing of such applications becomes repetitive and time-consuming due to frequent regression cycles. To overcome this limitation, an automation testing framework has been developed.

This automation framework helps in:

- Reducing manual testing effort
- Increasing execution speed of test cases
- Improving accuracy of regression testing
- Ensuring consistent validation of core functionalities

The framework is built using Selenium WebDriver for browser automation and TestNG for test execution and reporting.

□ 3. Application Under Test (AUT)

◆ Application Name:

OrangeHRM Open Source Demo Application

◆ URL:

<https://opensource-demo.orangehrmlive.com/>

◆ Type:

Web-based Human Resource Management System

◆ Key Functional Areas:

- User Authentication (Login/Logout)
 - Employee Management
 - Leave Management
 - Dashboard Navigation
 - Password Recovery (Forgot Password)
-

□ 4. Project Objectives

The primary objective of this automation project is to validate the core functionalities of the OrangeHRM application through automated test scripts.

🎯 Primary Objectives:

1. Login Functionality Testing

- Validate login with correct credentials (Admin/admin123)
- Validate login with invalid credentials
- Validate error messages for empty username/password fields

2. Forgot Password Functionality Testing

- Verify password reset with valid username
- Validate system behavior for invalid username
- Verify warning messages and success messages

3. Navigation Testing

- Validate login → dashboard navigation
- Verify navigation to Apply Leave page via menu
- Ensure UI elements are accessible after login

4. Page Validation Testing

- Verify page headers
 - Validate system messages
 - Ensure correct page transitions
-

□ 5. Scope of the Project

✓ In Scope (Covered in Automation Framework)

- Login module automation
 - Forgot password workflow automation
 - Dashboard page validation
 - Navigation to Apply Leave page
 - UI message validation
 - Functional testing using assertions
-

✗ Out of Scope (Not Covered)

- Performance testing (load/stress testing)
 - Security penetration testing
 - Database validation
 - API testing
 - Mobile application testing
-

□ 6. Automation Framework Architecture

The framework is designed using **Page Object Model (POM)** structure for better modularity.

■ Framework Layers:

◆ 1. Page Layer (src/main/java/pages)

Contains all page-specific classes representing UI pages.

- **HomePage.java**
 - Handles login operations
 - Username/password input
 - Login button click
 - Error message validation
- **DashBoardPage.java**
 - Handles post-login dashboard actions
 - Navigation to modules like Leave section
- **ApplyLeavePage.java**
 - Handles leave application functionality
 - Leave type selection
 - Date input handling
 - Apply button execution
- **ForgotPasswordPage.java**
 - Handles password reset functionality
 - Username input
 - Reset and cancel actions

- Warning message validation
 - **SuccessForgotPasswordPage.java**
 - Handles success confirmation screen
 - Validates success messages after reset request
-

◆ 2. Utilities Layer (src/main/java/utills)

Contains reusable constant values.

- **LoginMessages.java**
 - Stores login validation messages such as:
 - Welcome message
 - Invalid credentials message
 - Empty username/password messages
 - **ForgotPasswordMessages.java**
 - Stores forgot password validation messages:
 - Success message
 - Warning messages
 - HR contact messages
-

◆ 3. Test Layer (src/test/java)

Contains TestNG test classes.

- **BaseTest.java**
 - Initializes WebDriver
 - Launches browser
 - Opens application URL
 - Closes browser after execution
 - **LoginTests.java**
 - Validates login scenarios
 - **ForgotPasswordTests.java**
 - Validates password recovery functionality
 - **NavigationTest.java**
 - Validates navigation from dashboard to Apply Leave page
 - **LoadPageTest.java**
 - (Optional) Validates page load behavior
-

□ 7. Framework Design Pattern

The framework follows:

✓ Page Object Model (POM)

Each web page is represented as a separate Java class containing:

- Web elements (locators)
- Page actions (methods)

✓ TestNG Framework

Used for:

- Test execution
 - Test grouping
 - Assertions
 - Test suite management (testng.xml)
-

□ 8. Test Execution Flow

The execution flow of the automation framework is:

```
testng.xml
↓
BaseTest (Browser setup)
↓
Test Class (Login / Navigation / Forgot Password)
↓
Page Object Classes
↓
Selenium WebDriver Actions
↓
OrangeHRM Application
↓
Assertion Validation (TestNG)
```

□ 9. Key Features of Framework

- Cross-browser testing support (Chrome & Firefox)
- Modular Page Object Model design
- Reusable test methods
- Centralized message validation (utils package)
- TestNG-based execution control
- Maintainable and scalable structure

□ 10. Project Outcome

After completion of this automation framework:

- Core modules of OrangeHRM are successfully automated
 - Test execution is faster compared to manual testing
 - Framework supports regression testing
 - Code is reusable and easy to maintain
 - Ready for CI/CD integration in future enhancements
-